arima: employee's wage time series

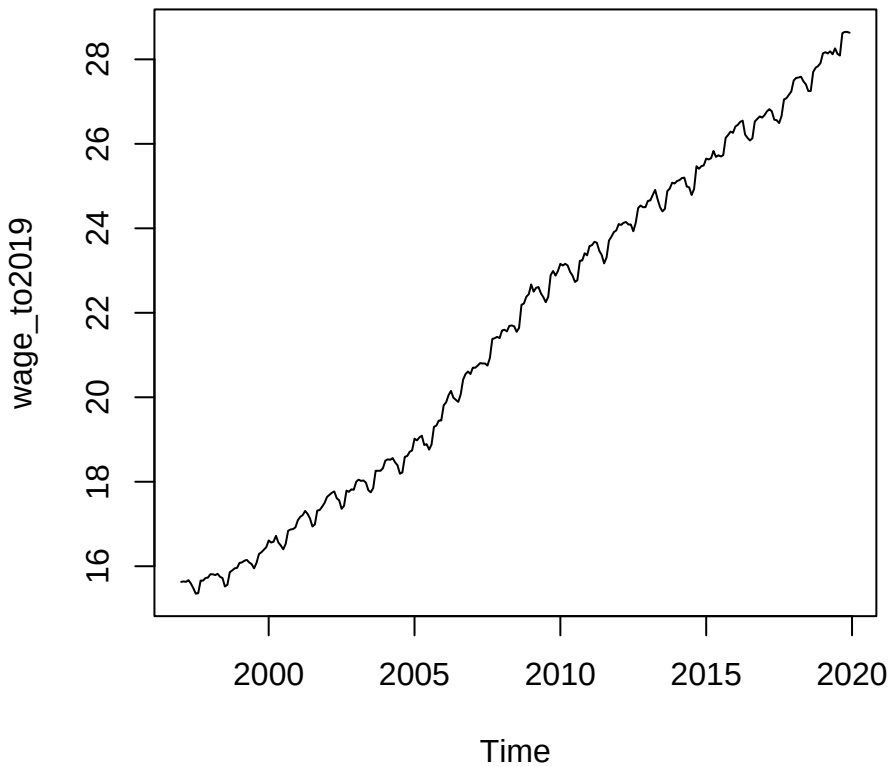
2026 March 16

Average hourly wage rates in Canada from January 1997 to November 2024 (monthly), computed over all industry classes. The wage rate includes both full-time and part-time employees of both sexes, and only those aged 15 or older are considered.

Compare additive seasonal exponential smoothing, seasonal ARIMA with differencing at lag 12, ARIMAX regression on linear and sinusoid.

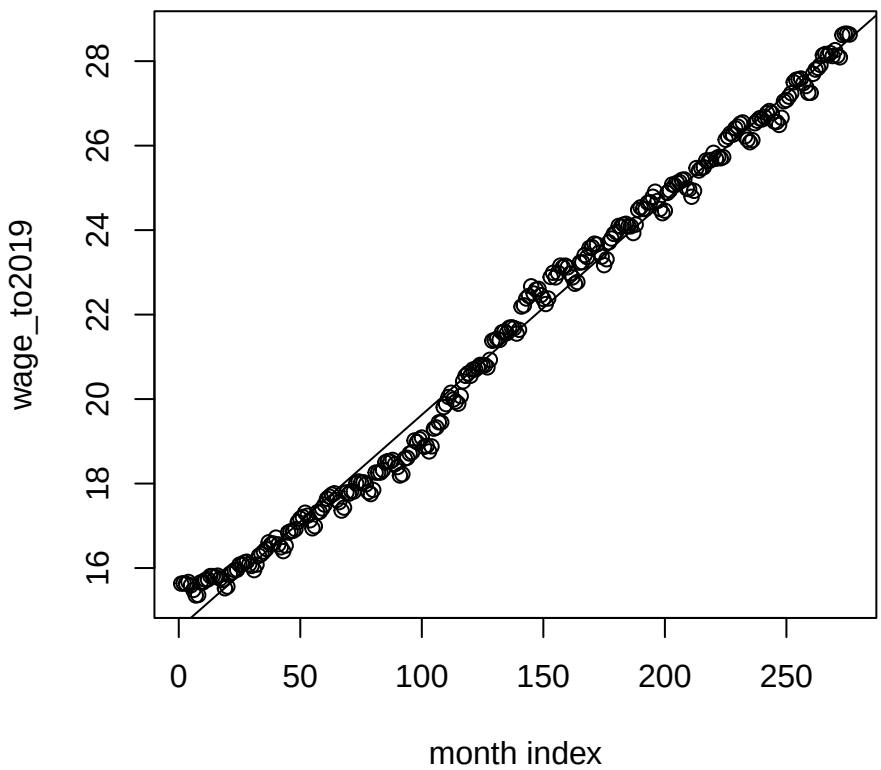
Visualization

```
w = read.csv("employee_wages_total_industry.csv",header=T)

#28 years = 336 months

wagerate = ts(w$employee_wages,start=c(1997,1),frequency=12)
# Subset to Dec 2019 (avoid pandemic period)
wage_to2019 = window(wagerate,start=c(1997,1),end=c(2019,12))

summary(wage_to2019)
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   15.35   17.77   21.68   21.57   25.07   28.65

# 23 years, 276 months
ntotal = length(wage_to2019)
print(ntotal)
#> [1] 276

ntrain = 18*12 # 216
train = window(wagerate,start=c(1997,1),end=c(2014,12))
holdout = window(wagerate,start=c(2015,1),end=c(2019,12))

plot(wage_to2019)
```




```
tt = 1:276
reg = lm(wage_to2019~tt)
summary(reg)
#>
#> Call:
#> lm(formula = wage_to2019 ~ tt)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -1.01882 -0.22855  0.01142  0.23853  1.01151
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept) 1.457e+01  4.471e-02   325.8   <2e-16 ***
#> tt          5.059e-02  2.798e-04   180.8   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.3704 on 274 degrees of freedom
#> Multiple R-squared:  0.9917, Adjusted R-squared:  0.9917
#> F-statistic: 3.268e+04 on 1 and 274 DF,  p-value: < 2.2e-16
#Coefficients:
#             Estimate Std. Error t value Pr(>|t|)
```

```
#(Intercept) 1.457e+01  4.471e-02   325.8   <2e-16 ***
#tt          5.059e-02  2.798e-04   180.8   <2e-16 ***
#Residual standard error: 0.3704 on 274 degrees of freedom
#Multiple R-squared:  0.9917,    Adjusted R-squared:  0.9917

plot(tt,wage_to2019,xlab="month index")
abline(reg$coefficient)
```



Forecast method 1: Winters additive seasonal

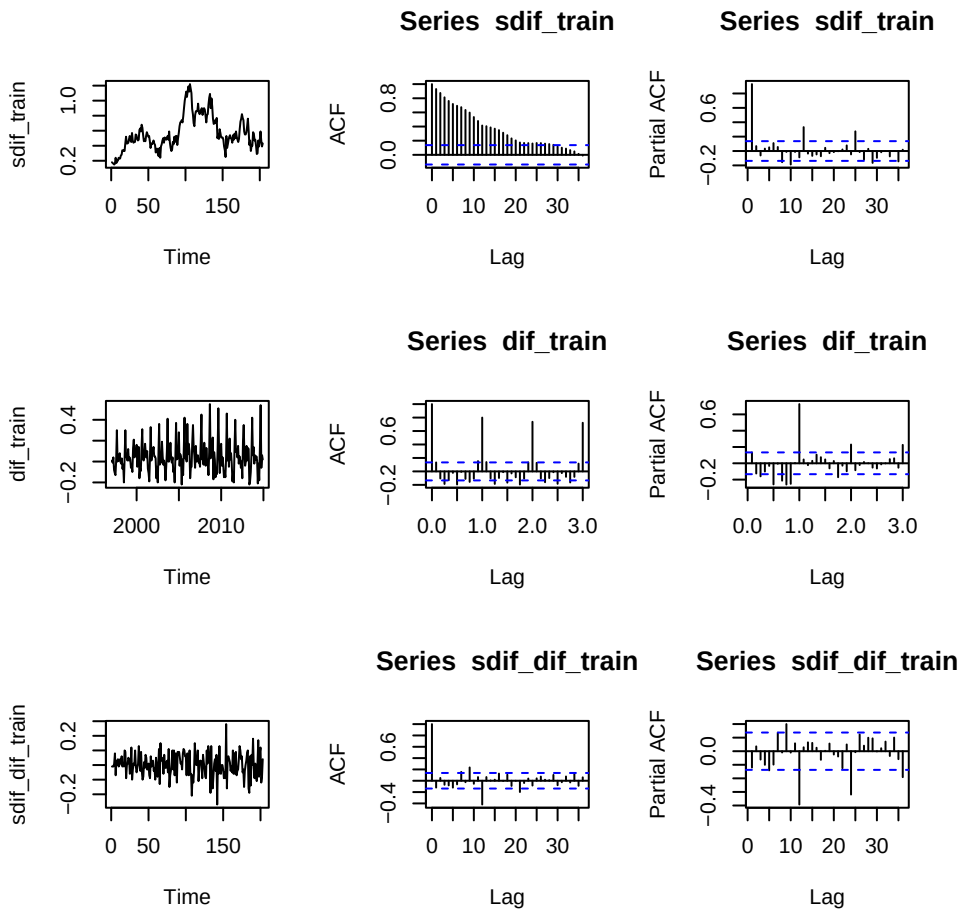
```
#' @description forecast rule: Winters additive seasonal exponential smoothing
#' @param train training set (ts object) with length ntrain
#' @param holding holdout set with length nholdout
#' @param alpha parameter for level equation
#' @param beta parameter for slope equation
#' @param gamma parameter for season equation
#' @param level last ell value in training set
#' @param slope last b  value in training set
#' @param season vector of s value in the last period in training set,
#'         length(season) = m = periodicity
```

```r
#' @return list with holdout set rmse, holdout set forecast vector
aseason_fc = function(train, holdout, alpha, beta, gamma, level, slope, season)
{ nh = length(holdout)
  m = length(season)
  sse = 0
  fc_vec = numeric(nh)
  L = level; b = slope; s = season
  for (i in 1:nh)
  { fc = L + b + s[((i-1)%%m) + 1]
    fc_vec[i] = fc
    yt = holdout[i]
    fcerror = yt - fc
    sse = sse + fcerror^2
    L_old = L
    L = alpha * (yt - s[((i-1)%%m) + 1]) + (1-alpha) * (L_old + b)
    b = beta * (L-L_old) + (1-beta) * b
    s[((i-1)%%m) + 1] = gamma * (yt-L_old-b) + (1-gamma) * s[((i-1)%%m) + 1]
  }
  return(list(rmse=sqrt(sse/nh), fc=fc_vec))
}

wa = HoltWinters(train,seasonal="additive")

alpha = wa$alpha;  beta = wa$beta; gamma = wa$gamma
level = wa$coef[1];  slope = wa$coef[2]; season = wa$coefficient[3:14]

cat(alpha, beta, gamma,"\n")
#> 0.7939212 0.04253467 1
cat(level, slope,"\n")
#> 25.43346 0.04448807
print(season)
#>          s1          s2          s3          s4          s5          s6
#>  0.14388415  0.11842165  0.13854889  0.11937925 -0.08331720 -0.17076943
#>          s7          s8          s9         s10         s11         s12
#> -0.33813258 -0.23516736  0.17064524  0.12602303  0.12799822  0.05654466


# See notes on ARIMA representation of additive seasonal exponential smoothing.
# If alpha, beta, gamma are positive, then the ARIMA model
# has differencing at lag 1 and at lag 12 (frequency=12)

aseason_obj = aseason_fc(train,holdout,alpha,beta,gamma,level,slope,season)

aseason_rmse = as.numeric(aseason_obj$rmse)
print(aseason_rmse)
#> [1] 0.1047314
```

Forecast method 2: ARIMA

```r
#' @description
#' Moving 1-step ahead forecasts for holdout set, seasonal ARIMA
#' @param tsdata vector with training and holdout data
```

```r
#' @param ntrain length of training set
#' @param order ARIMA order using for training set
#' @param method as used for arima applied to training set
#' @param traincoef the $coef component for arima applied to training set
#' @param include.mean flag as used for arima applied to training set
#' @param xreg matrix with predictors for arima()
#' @return holdout set rmse and 1-step forecasts
arima_fc = function(tsdata,ntrain,order,seasonal,method,traincoef,include.mean,
   iprint=F)
{  #obj = arima(tsdata,order=order,seasonal=seasonal,init=traincoef,fixed=traincoef,
  # next line for debugging, after debugging, replace with above line
  obj <<- arima(tsdata,order=order,seasonal=seasonal,init=traincoef,fixed=traincoef,
     method=method, include.mean=include.mean,
     optim.control=list(maxit=0))
  # yt = y_{t|t-1} + innov ; fc = yt-innov
  fc = tsdata - obj$residuals
  ntotal = length(tsdata)
  holdout_fc = fc[(ntrain+1):ntotal]
  holdout = tsdata[(ntrain+1):ntotal]
  if(iprint) print(cbind(holdout,holdout_fc))
  rmse = sqrt(mean((holdout-holdout_fc)^2))
  list(rmse=rmse, fc=holdout_fc)
}

# seasonal difference at lag 12 (12 months_
sdif_train = train[-(1:12)] - train[-((ntrain-11):ntrain)]
# difference at lag 1
dif_train = diff(train)
# seasonal difference lag 12 after difference at lag 1
sdif_dif_train = dif_train[-(1:12)] - dif_train[-((ntrain-12):(ntrain-1))]

par(mfrow=c(3,3))
plot.ts(sdif_train)
acf(sdif_train,lag.max=36)
pacf(sdif_train,lag.max=36)
plot(dif_train)
acf(dif_train,lag.max=36)
pacf(dif_train,lag.max=36)
plot.ts(sdif_dif_train)
acf(sdif_dif_train,lag.max=36)
pacf(sdif_dif_train,lag.max=36)
```

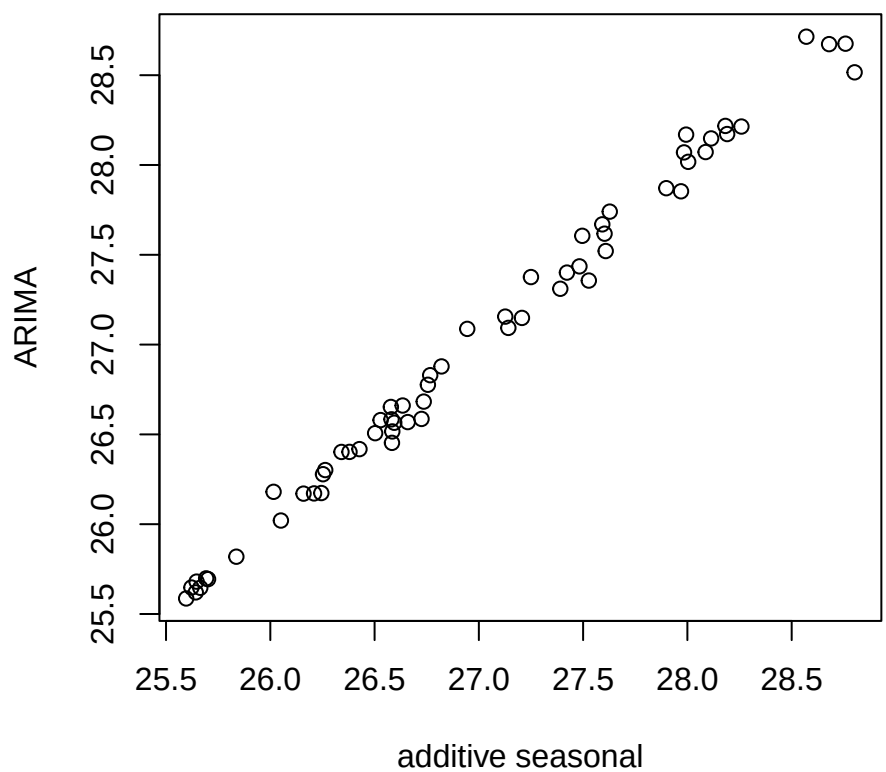



```r
# Differencing at lag 12, resulting series appears non-stationary.

library(forecast)
wage_auto = auto.arima(train,stationary=F,seasonal=T)

print(wage_auto)
#> Series: train
#> ARIMA(0,1,0)(0,1,1)[12]
#>
#> Coefficients:
#>          sma1
#>       -0.7895
#> s.e.   0.0573
#>
#> sigma^2 = 0.004089:  log likelihood = 265.17
#> AIC=-526.35   AICc=-526.29   BIC=-519.72

wage_arima = arima(train,order=c(0,1,0), seasonal=list(order=c(0,1,1)))
print(wage_arima)
#>
#> Call:
#> arima(x = train, order = c(0, 1, 0), seasonal = list(order = c(0, 1, 1)))
#>
```

```
#> Coefficients:
#>          sma1
#>       -0.7895
#> s.e.   0.0573
#>
#> sigma^2 estimated as 0.004054:  log likelihood = 265.17,  aic = -526.35

# holdout set rmse

arimafc_obj = arima_fc(wage_to2019,ntrain,order=c(0,1,0),
   seasonal=list(order=c(0,1,1)),method="ML",traincoef=wage_arima$coef,
   include.mean=F, iprint=F)

print(arimafc_obj$rmse)
#> [1] 0.06968725
```

Compare two forecast methods

```
compare2 =  cbind(aseason_obj$fc, arimafc_obj$fc)
colnames(compare2) = c("additive seasonal","ARIMA")
plot(compare2)
```



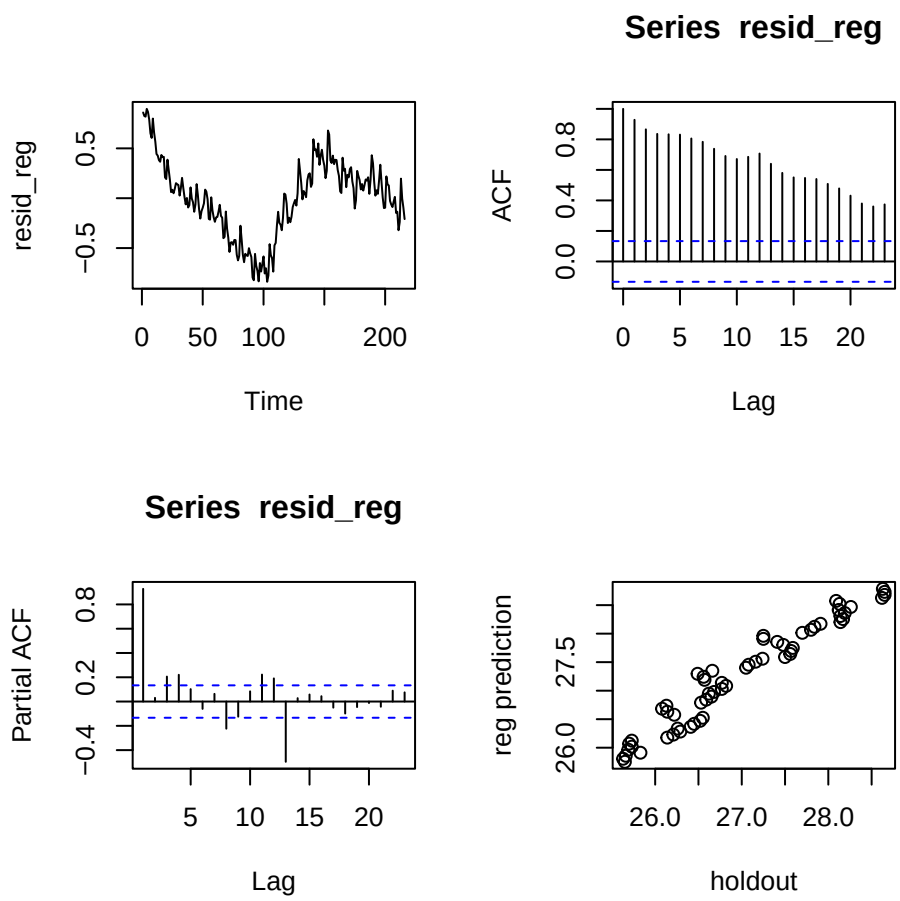
```r
# Debugging/validation
head(obj$residuals)
#>              Jan         Feb         Mar         Apr         May         Jun
#> 1997 0.009023982 0.004043390 0.002638565 0.002001943 0.001524632 0.001167381

print(obj$residuals[(ntrain-5):ntrain])
#> [1] -0.04621596  0.02660539  0.12486307 -0.11418084 -0.01054150  0.01087479

print(ntrain)
#> [1] 216

head(wage_arima$residuals)
#>              Jan         Feb         Mar         Apr         May         Jun
#> 1997 0.009023982 0.004043390 0.002638565 0.002001943 0.001524632 0.001167381

tail(wage_arima$residuals)
#>              Jul         Aug         Sep         Oct         Nov         Dec
#> 2014 -0.04621596  0.02660539  0.12486307 -0.11418084 -0.01054150  0.01087479
```

Forecast method 3: Regress on sinusoid and linear term, and ARIMAX

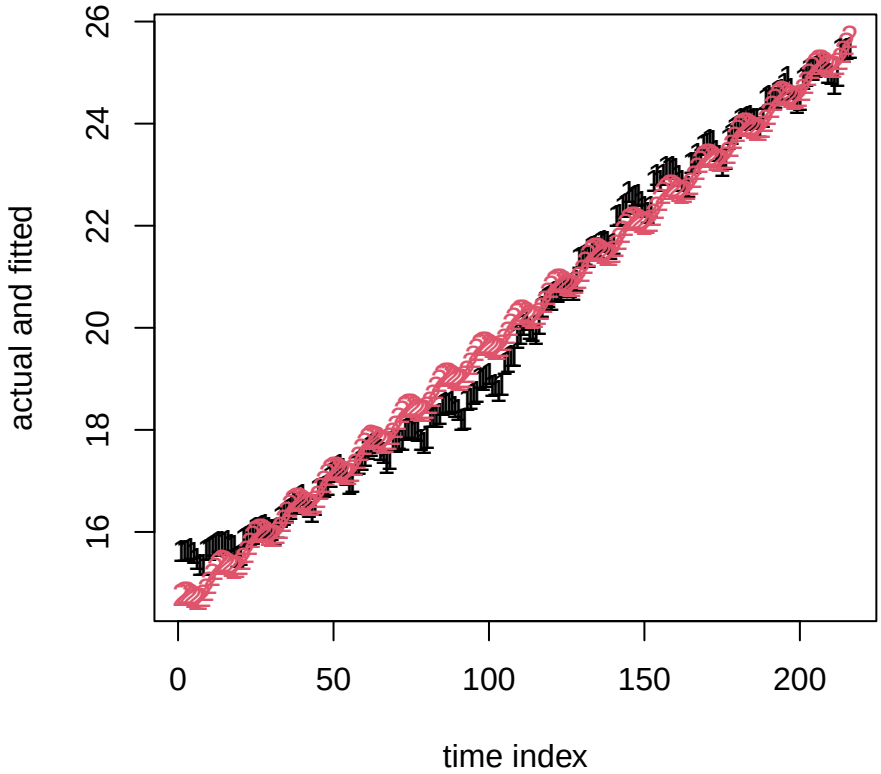
```r
#' @description
#' Moving 1-step ahead forecasts for holdout set, ARIMAX
#' @param tsdata vector with training and holdout data
#' @param ntrain length of training set
#' @param order ARIMA order using for training set
#' @param method as used for arima applied to training set
#' @param traincoef the $coef component for arima applied to training set
#' @param include.mean flag as used for arima applied to training set
#' @param xreg matrix with predictors for arima()
#' @return holdout set rmse and 1-step forecasts
arimax_fc = function(tsdata,ntrain,order,method,traincoef,include.mean,xreg,
   iprint=F)
{ # obj = arima(tsdata,order=order,init=traincoef,fixed=traincoef,
  # next line for debugging, after debugging, replace with above line
  obj <<- arima(tsdata,order=order,init=traincoef,fixed=traincoef,
     method=method, include.mean=include.mean, xreg=xreg,
     optim.control=list(maxit=0))
  # yt = y_{t|t-1} + innov ; fc = yt-innov
  fc = tsdata - obj$residuals
  ntotal = length(tsdata)
  holdout_fc = fc[(ntrain+1):ntotal]
  holdout = tsdata[(ntrain+1):ntotal]
  if(iprint) print(cbind(holdout,holdout_fc))
  rmse = sqrt(mean((holdout-holdout_fc)^2))
  list(rmse=rmse, fc=holdout_fc)
}
```

```r
mon_train = rep(1:12,18)
mon_holdout = rep(1:12,5)
mon_all = rep(1:12,23)

t_train = 1:216
t_holdout = 217:276
t_all = 1:276

train_df = data.frame(wage=train, tt=t_train,
   cosine=cos(2*pi*mon_train/12), sine=sin(2*pi*mon_train/12))
holdout_df = data.frame(wage=holdout, tt=t_holdout,
   cosine=cos(2*pi*mon_holdout/365), sine=sin(2*pi*mon_holdout/365))
all_df = data.frame(wage=wage_to2019, tt=t_all,
   cosine=cos(2*pi*mon_all/365), sine=sin(2*pi*mon_all/365))

reg_tt_cossin = lm(wage ~ tt+cosine+sine, data=train_df)
summary(reg_tt_cossin)
#>
#> Call:
#> lm(formula = wage ~ tt + cosine + sine, data = train_df)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -0.83854 -0.20867  0.03391  0.22727  0.89636
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept) 1.453e+01  5.204e-02 279.211  < 2e-16 ***
#> tt          5.104e-02  4.159e-04 122.718  < 2e-16 ***
#> cosine      1.439e-01  3.665e-02   3.927 0.000116 ***
#> sine        1.287e-01  3.668e-02   3.508 0.000550 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.3808 on 212 degrees of freedom
#> Multiple R-squared:  0.9861, Adjusted R-squared:  0.986
#> F-statistic:  5030 on 3 and 212 DF,  p-value: < 2.2e-16

resid_reg = reg_tt_cossin$residuals

pred_holdout = predict(reg_tt_cossin, newdata=holdout_df)

reg_rmse = sqrt(mean((holdout-pred_holdout)^2))

print(reg_rmse)
#> [1] 0.3418962

# Much worse than ARIMA

par(mfrow=c(2,2))
plot.ts(resid_reg) # seasonal patterm
```

```r
acf(resid_reg)
pacf(resid_reg)
plot(c(holdout),c(pred_holdout), xlab="holdout",ylab="reg prediction")
```




```r
# Regression residuals series appears non-stationary, try ARIMAX with
# ARIMA or differencing for residual series.


par(mfrow=c(1,1))
matplot(t_train,cbind(train_df$wage,reg_tt_cossin$fitted), xlab="time index",
 ylab="actual and fitted")
```

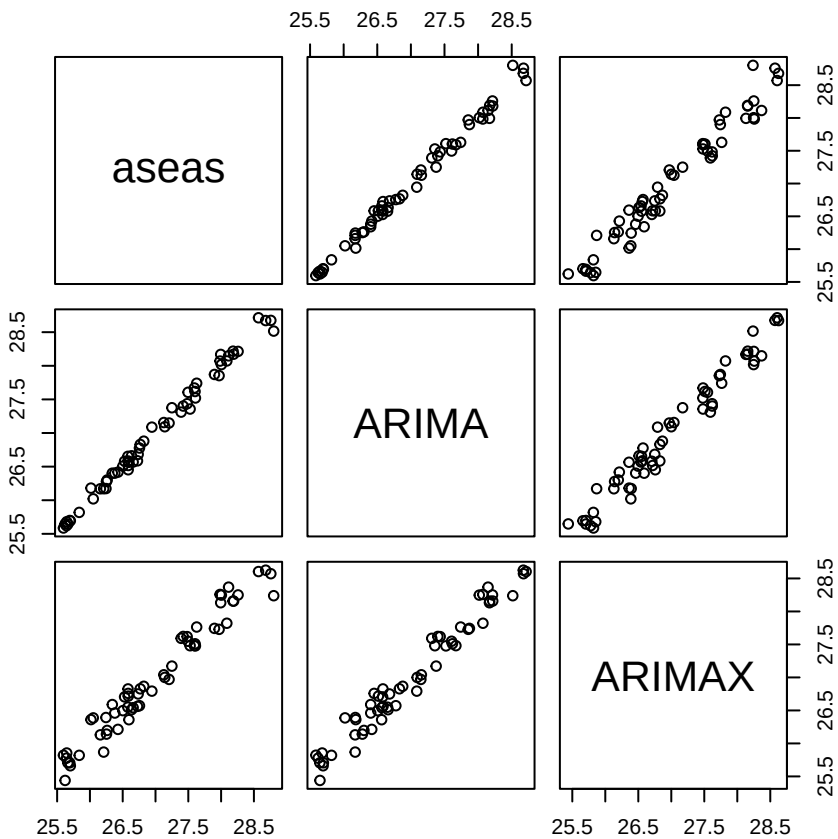



```
# Exact linear trend isn't a good match ?

resid_ts = ts(resid_reg, start=c(1,1), frequency=12)

# Try ARMAX with (1,0,0),(1,0,0)_{12}

resid_auto = auto.arima(resid_reg, stationary=F,seasonal=T)
print(resid_auto)
#> Series: resid_reg
#> ARIMA(5,1,1)
#>
#> Coefficients:
#>          ar1      ar2     ar3     ar4     ar5      ma1
#>       0.6784  -0.1324  0.0103  0.1197  0.2519  -0.9177
#> s.e.  0.0783   0.0801  0.0814  0.0828  0.0685   0.0471
#>
#> sigma^2 = 0.01352:  log likelihood = 160.17
#> AIC=-306.33   AICc=-305.79   BIC=-282.74

train_arimax_ML = arima(train_df$wage,order=c(5,1,1),include.mean=T,xreg=train_df[,2:4], method="ML")
print(train_arimax_ML) # no mean parameter so looks quite different from 2-step
#>
#> Call:
```

```
#> arima(x = train_df$wage, order = c(5, 1, 1), xreg = train_df[, 2:4], include.mean = T,
#>     method = "ML")
#>
#> Coefficients:
#>           ar1      ar2      ar3      ar4      ar5     ma1      tt  cosine
#>       -0.8103  -0.4794  -0.5306  -0.3871  -0.0301  0.5974  0.0465  0.1487
#> s.e.   0.1656   0.0926   0.1014   0.1003   0.0819  0.1503  0.0039  0.0137
#>         sine
#>       0.1121
#> s.e.  0.0138
#>
#> sigma^2 estimated as 0.01318:  log likelihood = 159.92,  aic = -299.85

train_arimax = arima(train_df$wage,order=c(5,1,1),include.mean=T,xreg=train_df[,2:4], method="CSS")
print(train_arimax)
#>
#> Call:
#> arima(x = train_df$wage, order = c(5, 1, 1), xreg = train_df[, 2:4], include.mean = T,
#>     method = "CSS")
#>
#> Coefficients:
#>          ar1      ar2     ar3     ar4     ar5      ma1      tt  cosine    sine
#>       0.6674  -0.1267  0.0033  0.1175  0.2559  -0.9327  0.0511  0.1470  0.1092
#> s.e.  0.0750   0.0792  0.0806  0.0826  0.0685   0.0419  0.0072  0.0117  0.0120
#>
#> sigma^2 estimated as 0.01302:  part log likelihood = 161.65

# CSS estimation is closer to 2-step estimation

xreg = arimax_fc(all_df$wage, ntrain=ntrain, order=c(5,1,1), method="CSS",
  traincoef=train_arimax$coef, include.mean=T, xreg=all_df[,2:4])

print(xreg$rmse)
#> [1] 0.1732654

# Vector of 3 rmses

rmse_vec = c(aseason_rmse, arimafc_obj$rmse, xreg$rmse)
print(round(rmse_vec,3))
#> [1] 0.105 0.070 0.173

# Best is ARIMA with difference at lag 1 and lag 12

# Do the RMSE value indicate adequate forecasts?

print(holdout)
#>        Jan   Feb   Mar   Apr   May   Jun   Jul   Aug   Sep   Oct   Nov   Dec
#> 2015 25.65 25.63 25.66 25.83 25.69 25.73 25.70 25.73 26.14 26.21 26.29 26.26
#> 2016 26.41 26.45 26.52 26.55 26.22 26.14 26.08 26.13 26.53 26.59 26.65 26.62
#> 2017 26.68 26.77 26.82 26.77 26.57 26.56 26.49 26.66 27.05 27.08 27.16 27.24
#> 2018 27.50 27.56 27.57 27.59 27.48 27.41 27.25 27.25 27.70 27.80 27.84 27.91
#> 2019 28.14 28.17 28.14 28.19 28.12 28.26 28.13 28.09 28.62 28.65 28.65 28.63
```

Compare forecasts of 3 methods

```
compare3 =  cbind(aseason_obj$fc, arimafc_obj$fc,xreg$fc)
colnames(compare3) = c("aseas","ARIMA", "ARIMAX")
pairs(compare3)
```



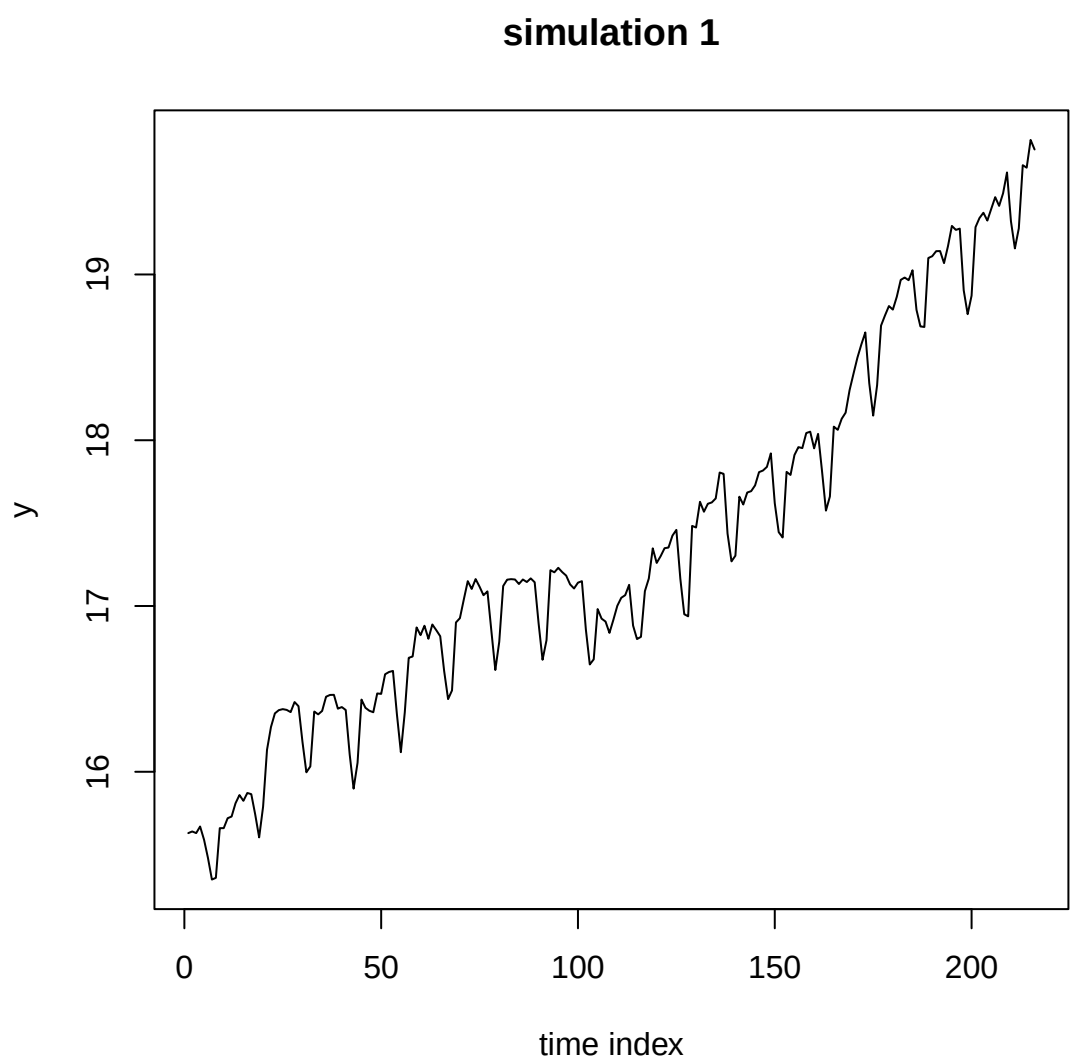
Prepare for simulation: Θ_1 and σ_ϵ^2 are the two parameters in the fitted seasonal ARIMA model.

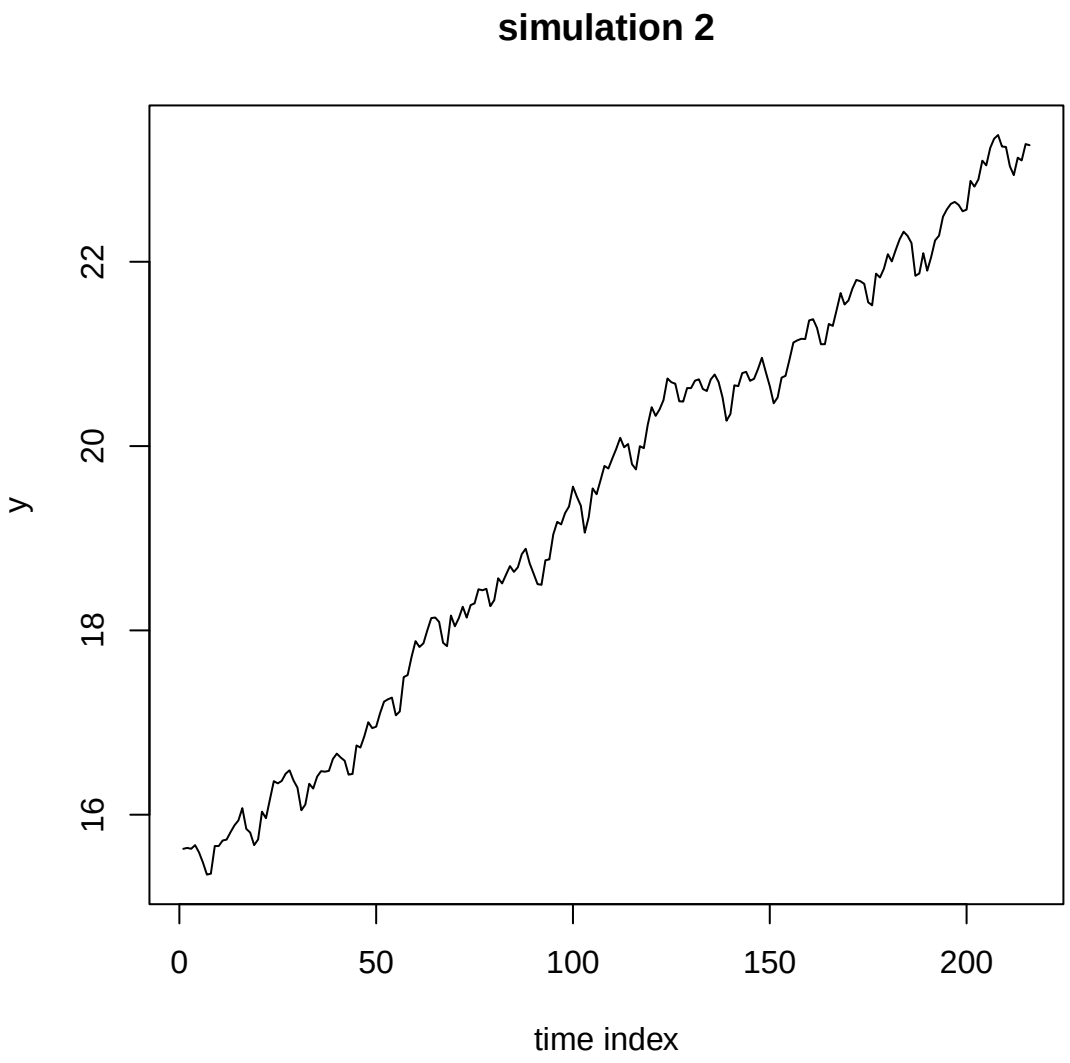
$$(1-B^{12})(1-B)Y_t = (1+\Theta_1 B^{12})\epsilon_t$$
$$Y_t - Y_{t-1} - (Y_{t-12} - Y_{t-13}) = \epsilon_t + \Theta_1\epsilon_{t-12}$$
$$Y_t = Y_{t-1} + (Y_{t-12} - Y_{t-13}) + \epsilon_t + \Theta_1\epsilon_{t-12}$$
$$Y_{14} = Y_{13} + (Y_2 - Y_1) + \epsilon_{14} + \Theta_1\epsilon_2$$
$$Y_{15} = Y_{14} + (Y_3 - Y_2) + \epsilon_{15} + \Theta_1\epsilon_3$$
$$Y_{16} = Y_{15} + (Y_4 - Y_3) + \epsilon_{16} + \Theta_1\epsilon_4$$
$$\cdots$$
$$Y_{25} = Y_{24} + (Y_{13} - Y_{12}) + \epsilon_{25} + \Theta_1\epsilon_{13}$$
$$Y_{26} = Y_{25} + (Y_{14} - Y_{13}) + \epsilon_{26} + \Theta_1\epsilon_{14}$$

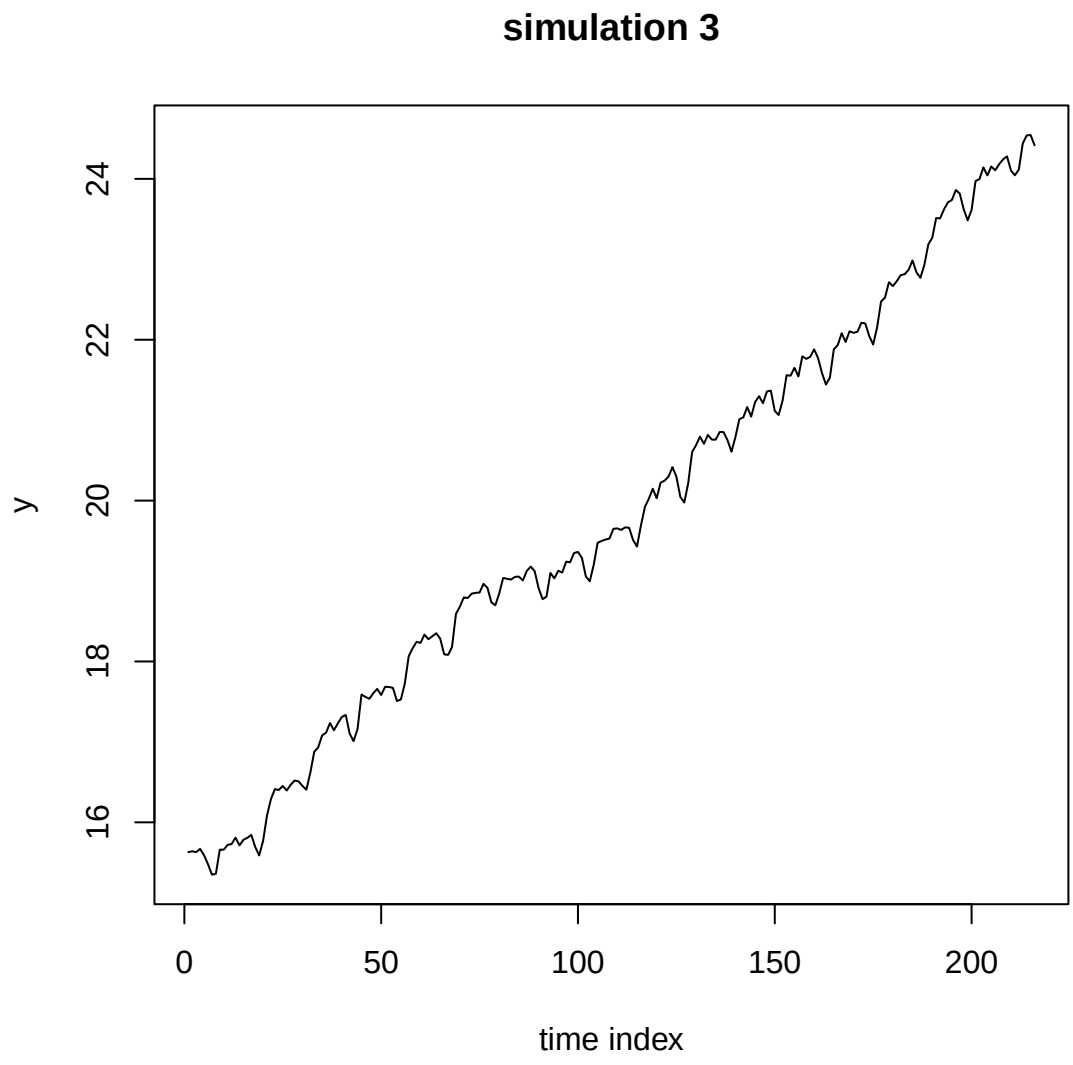
Simulation from ARIMA(0,1,0),(0,1,1)_12

```
sma1 = wage_arima$coef
sigma = sqrt(wage_arima$sigma)

set.seed(12345)
# Next line seems to fail to simulate from ARIMA(0,1,0),(0,1,1)_12 model
# y = arima.sim(n=216, model=list(order=c(0,1,0),
#  seasonal=list(order=c(0,1,1),frequency=12,ma=sma1)), sd=sigma)

ystart = wagerate[1:13]
ntrain = 216 # 18 years

par(mfrow=c(1,1))
plot(train, main="wages in training set")
```

```r
set.seed(123)
# Compare 3 simulations
for(isim in 1:3)
{ zma = rep(0,ntrain)
  # Generate 12 MA(1) sequences
  for(mon in 1:12)
  { i12 = seq(mon,ntrain,12)
    zma[i12] = arima.sim(n=18, model=list(ma=c(sma1)), sd=sigma)
  }
  # simulate from seasonal ARIMA
  y = rep(0,ntrain)
  y[1:13] = ystart
  for(i in 14:ntrain)
  { y[i] = y[i-1] + y[i-12] - y[i-13] + zma[i] }
  par(mfrow=c(1,1))
  plot.ts(y, xlab="time index", main=paste("simulation",isim))
}
```




simulation 2
y
16
18
20
22
0
50
100
150
200
time index




```
# ARIMA model is OK for forecasting but doesn't always generate series that
# look like actual data series. Some simulations can look quite different
# from the above. Run the above code with a different seed.
```